

```
from google.colab import files
uploaded=files.upload()
print(uploaded)
```

Choose files proj1.csv

proj1.csv(text/csv) - 5556 bytes, last modified: 15/06/2026 - 100% done

Saving proj1.csv to proj1 (1).csv

{'proj1 (1).csv': b'06.15 6:34 AM\nStudent_ID,Name,Age,Gender,Department,Study_Hours_Per_Day,Attendance_Percentage,Assignments_C

```
# =====
# PROJECT 2: Exploratory Data Analysis (EDA) using Python
# =====
# Dataset: Global Superstore Sales Dataset (Simulated)
# Objective: Understand sales patterns, customer behavior,
#           and product performance through EDA
# =====
```

```
# _____
# SECTION 1: Import Libraries
# _____
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
# Set global plot styles
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.family'] = 'DejaVu Sans'
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12
plt.rcParams['xtick.labelsize'] = 10
plt.rcParams['ytick.labelsize'] = 10
```

```
sns.set_style("whitegrid")
sns.set_palette("husl")
```

```
print("=" * 60)
print("    EXPLORATORY DATA ANALYSIS - GLOBAL SUPERSTORE")
print("=" * 60)
print("✅ All libraries imported successfully!")
```

```
=====
    EXPLORATORY DATA ANALYSIS - GLOBAL SUPERSTORE
=====
✅ All libraries imported successfully!
```

```
# _____
# SECTION 2: Dataset Creation (Simulated)
# _____
```

```
np.random.seed(42)
n = 1000 # Number of records
```

```
# Define categories and regions
categories = ['Technology', 'Furniture', 'Office Supplies']
sub_cats = {
    'Technology' : ['Phones', 'Computers', 'Accessories', 'Copiers'],
    'Furniture' : ['Chairs', 'Tables', 'Bookcases', 'Furnishings'],
    'Office Supplies': ['Binders', 'Paper', 'Storage', 'Appliances']
}
regions = ['North', 'South', 'East', 'West']
segments = ['Consumer', 'Corporate', 'Home Office']
ship_modes = ['Standard Class', 'Second Class', 'First Class', 'Same Day']
```

```
# Assign categories
cat_choices = np.random.choice(categories, n, p=[0.35, 0.30, 0.35])
```

```

# Assign sub-categories based on category
sub_cat_list = [np.random.choice(sub_cats[c]) for c in cat_choices]

# Generate numerical features
sales = np.where(
    cat_choices == 'Technology',
    np.random.exponential(scale=500, size=n),
    np.where(
        cat_choices == 'Furniture',
        np.random.exponential(scale=300, size=n),
        np.random.exponential(scale=150, size=n)
    )
)
sales = np.clip(sales, 10, 5000)

discount = np.random.choice(
    [0.0, 0.1, 0.2, 0.3, 0.4, 0.5],
    n,
    p=[0.3, 0.25, 0.20, 0.15, 0.07, 0.03]
)

# Profit influenced by discount
profit = sales * (0.25 - discount * 0.8) + np.random.normal(0, 20, n)

quantity = np.random.randint(1, 15, n)
shipping_days = np.random.randint(1, 8, n)

# Categorical features
regions_col = np.random.choice(regions, n)
segments_col = np.random.choice(segments, n, p=[0.50, 0.30, 0.20])
ship_col = np.random.choice(ship_modes, n, p=[0.60, 0.20, 0.15, 0.05])

# Dates
dates = pd.date_range(start='2021-01-01', end='2023-12-31', periods=n)

# Introduce missing values (~3%)
sales_with_na = sales.copy().astype(float)
profit_with_na = profit.copy().astype(float)
missing_idx_s = np.random.choice(n, int(n * 0.03), replace=False)
missing_idx_p = np.random.choice(n, int(n * 0.02), replace=False)
sales_with_na[missing_idx_s] = np.nan
profit_with_na[missing_idx_p] = np.nan

# Introduce outliers
outlier_idx = np.random.choice(n, 15, replace=False)
sales_with_na[outlier_idx] = np.random.uniform(4500, 5000, 15)

# Build DataFrame
df = pd.DataFrame({
    'Order_Date' : dates,
    'Category' : cat_choices,
    'Sub_Category' : sub_cat_list,
    'Region' : regions_col,
    'Segment' : segments_col,
    'Ship_Mode' : ship_col,
    'Sales' : sales_with_na,
    'Profit' : profit_with_na,
    'Quantity' : quantity,
    'Discount' : discount,
    'Shipping_Days' : shipping_days
})

df['Year'] = df['Order_Date'].dt.year
df['Month'] = df['Order_Date'].dt.month
df['Month_Name'] = df['Order_Date'].dt.strftime('%b')

print("✅ Dataset created successfully!")
print(f"Shape: {df.shape[0]} rows x {df.shape[1]} columns\n")
print(df.head(10).to_string())

```

✅ Dataset created successfully!
Shape: 1000 rows x 14 columns

	Order_Date	Category	Sub_Category	Region	Segment	Ship_Mode	Sales	Profit	Quantity
0	2021-01-01 00:00:00.000000000	Furniture	Bookcases	West	Consumer	Second Class	NaN	34.373661	1
1	2021-01-02 02:16:56.216216216	Office Supplies	Appliances	West	Consumer	Same Day	243.409108	41.925816	
2	2021-01-03 04:33:52.432432432	Office Supplies	Paper	East	Consumer	Standard Class	883.764598	219.993735	1
3	2021-01-04 06:50:48.648648648	Furniture	Furnishings	South	Consumer	Standard Class	937.391300	202.704290	

4	2021-01-05 09:07:44.864864864	Technology	Copiers	North	Consumer	Second Class	239.175682	21.851888	1
5	2021-01-06 11:24:41.081081081	Technology	Accessories	East	Corporate	Standard Class	254.432757	52.392971	1
6	2021-01-07 13:41:37.297297297	Technology	Copiers	East	Corporate	Standard Class	433.955527	-1.668413	
7	2021-01-08 15:58:33.513513513	Office Supplies	Storage	East	Consumer	Standard Class	93.825948	24.406356	
8	2021-01-09 18:15:29.729729729	Furniture	Chairs	North	Corporate	Standard Class	563.134143	45.700636	
9	2021-01-10 20:32:25.945945946	Office Supplies	Binders	North	Consumer	Second Class	10.000000	-34.157878	

```
# -----
# SECTION 3: Dataset Overview
# -----

print("\n" + "=" * 60)
print("    SECTION 3: DATASET OVERVIEW")
print("=" * 60)

# 3.1 Basic Info
print("\n🚀 3.1 – Dataset Information")
print("-" * 40)
df.info()

# 3.2 Shape
print(f"\n🚀 3.2 – Dataset Shape")
print(f"    Rows    : {df.shape[0]}")
print(f"    Columns : {df.shape[1]}")

# 3.3 Data Types
print("\n🚀 3.3 – Data Types")
print(df.dtypes.to_string())

# 3.4 First & Last rows
print("\n🚀 3.4 – First 5 Rows")
print(df.head().to_string())
print("\n🚀 3.5 – Last 5 Rows")
print(df.tail().to_string())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Order_Date      1000 non-null   datetime64[ns]
1   Category        1000 non-null   object
2   Sub_Category    1000 non-null   object
3   Region          1000 non-null   object
4   Segment         1000 non-null   object
5   Ship_Mode       1000 non-null   object
6   Sales           970 non-null    float64
7   Profit          980 non-null    float64
8   Quantity        1000 non-null    int64
9   Discount        1000 non-null    float64
10  Shipping_Days   1000 non-null    int64
11  Year            1000 non-null    int32
12  Month           1000 non-null    int32
13  Month_Name      1000 non-null    object
dtypes: datetime64[ns](1), float64(3), int32(2), int64(2), object(6)
memory usage: 101.7+ KB
```

```
🚀 3.2 – Dataset Shape
Rows    : 1000
Columns : 14
```

```
🚀 3.3 – Data Types
Order_Date      datetime64[ns]
Category        object
Sub_Category    object
Region          object
Segment         object
Ship_Mode       object
Sales           float64
Profit          float64
Quantity        int64
Discount        float64
Shipping_Days   int64
```

1	2021-01-02 02:16:56.216216216	Office Supplies	Appliances	West	Consumer	Same Day	243.409108	41.925816	
2	2021-01-03 04:33:52.432432432	Office Supplies	Paper	East	Consumer	Standard Class	883.764598	219.993735	:
3	2021-01-04 06:50:48.648648648	Furniture	Furnishings	South	Consumer	Standard Class	937.391300	202.704290	
4	2021-01-05 09:07:44.864864864	Technology	Copiers	North	Consumer	Second Class	239.175682	21.851888	:

	Order_Date	Category	Sub_Category	Region	Segment	Ship_Mode	Sales	Profit	Q
995	2023-12-26 14:52:15.135135136	Technology	Phones	East	Corporate	First Class	338.957373	73.038798	
996	2023-12-27 17:09:11.351351360	Office Supplies	Storage	South	Consumer	Standard Class	100.466660	-12.034710	
997	2023-12-28 19:26:07.567567568	Technology	Phones	East	Consumer	Standard Class	1003.382409	18.590112	
998	2023-12-29 21:43:03.783783792	Office Supplies	Paper	North	Home Office	First Class	45.436214	-29.948785	
999	2023-12-31 00:00:00.000000000	Furniture	Furnishings	West	Consumer	Same Day	401.166798	24.878252	

```
# -----
# SECTION 4: Missing Value Analysis
# -----

print("\n" + "=" * 60)
print("    SECTION 4: MISSING VALUE ANALYSIS")
print("=" * 60)

# Calculate missing values
missing_count = df.isnull().sum()
missing_pct   = (df.isnull().sum() / len(df) * 100).round(2)
missing_df    = pd.DataFrame({
    'Missing Count'      : missing_count,
    'Missing Percentage': missing_pct
}).sort_values('Missing Count', ascending=False)

missing_df = missing_df[missing_df['Missing Count'] > 0]

print("\n🔥 Missing Value Summary:")
print(missing_df.to_string())

# --- Visualization -----
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Missing Value Analysis', fontsize=16, fontweight='bold', y=1.02)

# Heatmap of missing values
ax1 = axes[0]
missing_matrix = df[['Sales', 'Profit', 'Quantity',
                    'Discount', 'Shipping_Days']].isnull()
sns.heatmap(missing_matrix,
            cbar=True,
            yticklabels=False,
            cmap='RdYlGn_r',
            ax=ax1)
ax1.set_title('Missing Value Heatmap\n(Yellow = Missing)', fontweight='bold')
ax1.set_xlabel('Columns')

# Bar chart of missing counts
ax2 = axes[1]
cols    = missing_df.index
counts  = missing_df['Missing Count']
colors  = ['#e74c3c' if c > 20 else '#f39c12' for c in counts]

bars = ax2.bar(cols, counts, color=colors, edgecolor='white', linewidth=1.5)
ax2.set_title('Missing Value Count per Column', fontweight='bold')
ax2.set_xlabel('Column')
ax2.set_ylabel('Missing Count')

for bar, count, pct in zip(bars, counts, missing_df['Missing Percentage']):
    ax2.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.5,
             f'{count}\n({pct}%)',
             ha='center', va='bottom', fontweight='bold', fontsize=10)

plt.tight_layout()
plt.savefig('missing_values.png', dpi=150, bbox_inches='tight')
plt.show()

# Handle missing values
df['Sales'].fillna(df['Sales'].median(), inplace=True)
df['Profit'].fillna(df['Profit'].median(), inplace=True)

print("\n✅ Missing values handled:")
print(f"    Sales → filled with median: {df['Sales'].median():.2f}")
```

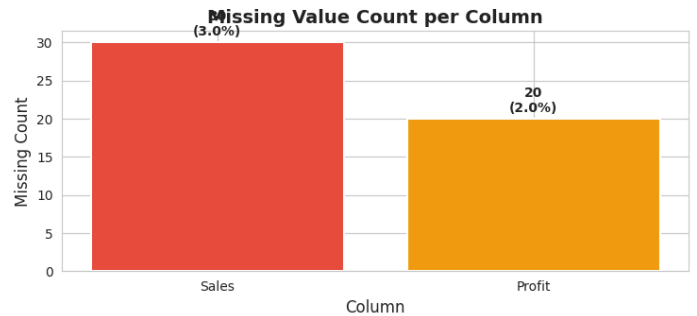
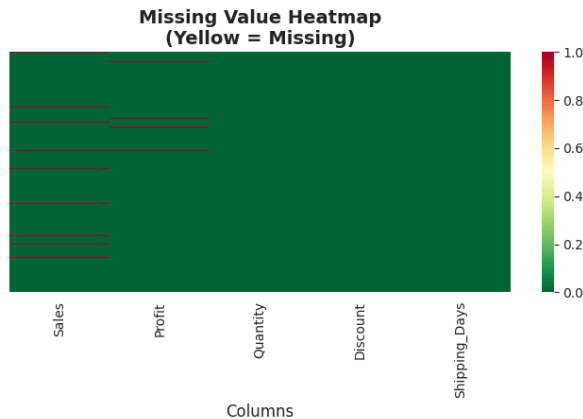
```
print(f" Profit → filled with median: {df['Profit'].median():.2f}")
print(f" Remaining missing values: {df.isnull().sum().sum()}")
```

SECTION 4: MISSING VALUE ANALYSIS

🔥 Missing Value Summary:

	Missing Count	Missing Percentage
Sales	30	3.0
Profit	20	2.0

Missing Value Analysis



✅ Missing values handled:

Sales → filled with median: 211.64

Profit → filled with median: 24.64

Remaining missing values: 0

```
# -----
# SECTION 5: Descriptive Statistics
# -----

print("\n" + "=" * 60)
print(" SECTION 5: DESCRIPTIVE STATISTICS")
print("=" * 60)

num_cols = ['Sales', 'Profit', 'Quantity', 'Discount', 'Shipping_Days']

# Extended statistics
stats_df = df[num_cols].describe().T
stats_df['median'] = df[num_cols].median()
stats_df['mode'] = df[num_cols].mode().iloc[0]
stats_df['skewness'] = df[num_cols].skew()
stats_df['kurtosis'] = df[num_cols].kurtosis()
stats_df['variance'] = df[num_cols].var()
stats_df['cv_%'] = (df[num_cols].std() / df[num_cols].mean() * 100).round(2)

print("\n🔥 Extended Descriptive Statistics:")
print(stats_df.round(2).to_string())

# — Pretty Summary Table —————
print("\n🔥 Key Metrics Summary:")
print("-" * 65)
print(f"{'Metric':<18} {'Sales':>10} {'Profit':>10} {'Quantity':>10} {'Discount':>10}")
print("-" * 65)

metrics = {
    'Mean' : df[['Sales', 'Profit', 'Quantity', 'Discount']].mean(),
    'Median' : df[['Sales', 'Profit', 'Quantity', 'Discount']].median(),
    'Std Dev' : df[['Sales', 'Profit', 'Quantity', 'Discount']].std(),
    'Min' : df[['Sales', 'Profit', 'Quantity', 'Discount']].min(),
    'Max' : df[['Sales', 'Profit', 'Quantity', 'Discount']].max(),
    'Skewness' : df[['Sales', 'Profit', 'Quantity', 'Discount']].skew(),
}
```

```

for metric, values in metrics.items():
    print(f"{metric:<18} {values['Sales']:>10.2f} {values['Profit']:>10.2f} "
          f"{values['Quantity']:>10.2f} {values['Discount']:>10.2f}")
print("-" * 65)

```

```

=====
SECTION 5: DESCRIPTIVE STATISTICS
=====

```

📌 Extended Descriptive Statistics:

	count	mean	std	min	25%	50%	75%	max	median	mode	skewness	kurtosis	variance	c
Sales	1000.0	391.98	652.03	10.00	81.83	211.64	456.78	4973.96	211.64	10.00	5.01	29.83	425146.43	166
Profit	1000.0	42.68	72.53	-292.95	-0.08	24.64	62.08	524.59	24.64	24.64	2.09	8.46	5259.99	169
Quantity	1000.0	7.65	4.14	1.00	4.00	8.00	11.00	14.00	8.00	1.00	-0.09	-1.24	17.14	54
Discount	1000.0	0.15	0.14	0.00	0.00	0.10	0.22	0.50	0.10	0.00	0.66	-0.47	0.02	90
Shipping_Days	1000.0	3.91	2.03	1.00	2.00	4.00	6.00	7.00	4.00	1.00	0.04	-1.25	4.12	51

📌 Key Metrics Summary:

Metric	Sales	Profit	Quantity	Discount
Mean	391.98	42.68	7.65	0.15
Median	211.64	24.64	8.00	0.10
Std Dev	652.03	72.53	4.14	0.14
Min	10.00	-292.95	1.00	0.00
Max	4973.96	524.59	14.00	0.50
Skewness	5.01	2.09	-0.09	0.66

```

# -----
# SECTION 6: Distribution Analysis
# -----

print("\n" + "=" * 60)
print("  SECTION 6: DISTRIBUTION ANALYSIS")
print("=" * 60)

fig = plt.figure(figsize=(18, 14))
fig.suptitle('Distribution Analysis of Numerical Variables',
             fontsize=18, fontweight='bold', y=1.01)

gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)

plot_configs = [
    ('Sales',      '#3498db', 'Sales Distribution (Right-Skewed)'),
    ('Profit',     '#2ecc71', 'Profit Distribution'),
    ('Quantity',   '#e74c3c', 'Quantity Distribution (Uniform)'),
    ('Discount',   '#9b59b6', 'Discount Distribution'),
    ('Shipping_Days', '#f39c12', 'Shipping Days Distribution'),
]

for idx, (col, color, title) in enumerate(plot_configs):
    row = idx // 3
    col_pos = idx % 3
    ax = fig.add_subplot(gs[row, col_pos])

    # Histogram with KDE
    data = df[col].dropna()
    ax.hist(data, bins=30, color=color, alpha=0.6,
            edgecolor='white', linewidth=0.5, density=True)

    # KDE overlay
    from scipy.stats import gaussian_kde
    kde = gaussian_kde(data)
    x_range = np.linspace(data.min(), data.max(), 300)
    ax.plot(x_range, kde(x_range), color=color, linewidth=2.5, label='KDE')

    # Vertical lines for mean and median
    ax.axvline(data.mean(), color='red', linestyle='--',
               linewidth=1.8, label=f'Mean: {data.mean():.1f}')
    ax.axvline(data.median(), color='navy', linestyle=':',
               linewidth=1.8, label=f'Median: {data.median():.1f}')

    ax.set_title(title, fontweight='bold', fontsize=11)
    ax.set_xlabel(col)
    ax.set_ylabel('Density')
    ax.legend(fontsize=8)

```

```

# Annotation box
skew_val = data.skew()
skew_txt = f'Skew: {skew_val:.2f}'
ax.text(0.97, 0.95, skew_txt,
        transform=ax.transAxes,
        ha='right', va='top',
        bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8),
        fontsize=9)

# Last subplot: Q-Q plot for Sales
ax_qq = fig.add_subplot(gs[1, 2])
stats.probplot(df['Sales'], dist="norm", plot=ax_qq)
ax_qq.set_title('Q-Q Plot: Sales\n(Normality Check)', fontweight='bold', fontsize=11)
ax_qq.get_lines()[0].set(color='#3498db', markersize=3, alpha=0.5)
ax_qq.get_lines()[1].set(color='red', linewidth=2)

plt.savefig('distributions.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n📊 Distribution Insights:")
for col in num_cols:
    skew = df[col].skew()
    if abs(skew) < 0.5: skew_type = "approximately symmetric"
    elif skew > 0      : skew_type = "right-skewed (positively skewed)"
    else               : skew_type = "left-skewed (negatively skewed)"
    print(f"    {col:<15} → Skewness: {skew:+.2f} → {skew_type}")

```

```

# -----
# SECTION 6: Distribution Analysis
# -----

print("\n" + "=" * 60)
print("    SECTION 6: DISTRIBUTION ANALYSIS")
print("=" * 60)

fig = plt.figure(figsize=(18, 14))
fig.suptitle('Distribution Analysis of Numerical Variables',
             fontsize=18, fontweight='bold', y=1.01)

gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)

plot_configs = [
    ('Sales',      '#3498db', 'Sales Distribution (Right-Skewed)'),
    ('Profit',     '#2ecc71', 'Profit Distribution'),
    ('Quantity',   '#e74c3c', 'Quantity Distribution (Uniform)'),
    ('Discount',   '#9b59b6', 'Discount Distribution'),
    ('Shipping_Days', '#f39c12', 'Shipping Days Distribution'),
]

for idx, (col, color, title) in enumerate(plot_configs):
    row = idx // 3
    col_pos = idx % 3
    ax = fig.add_subplot(gs[row, col_pos])

    # Histogram with KDE
    data = df[col].dropna()
    ax.hist(data, bins=30, color=color, alpha=0.6,
            edgecolor='white', linewidth=0.5, density=True)

    # KDE overlay
    from scipy.stats import gaussian_kde
    kde = gaussian_kde(data)
    x_range = np.linspace(data.min(), data.max(), 300)
    ax.plot(x_range, kde(x_range), color=color, linewidth=2.5, label='KDE')

    # Vertical lines for mean and median
    ax.axvline(data.mean(), color='red', linestyle='--',
               linewidth=1.8, label=f'Mean: {data.mean():.1f}')
    ax.axvline(data.median(), color='navy', linestyle=':',
               linewidth=1.8, label=f'Median: {data.median():.1f}')

    ax.set_title(title, fontweight='bold', fontsize=11)
    ax.set_xlabel(col)
    ax.set_ylabel('Density')
    ax.legend(fontsize=8)

```

```

# Annotation box
skew_val = data.skew()
skew_txt = f'Skew: {skew_val:.2f}'
ax.text(0.97, 0.95, skew_txt,
        transform=ax.transAxes,
        ha='right', va='top',
        bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8),
        fontsize=9)

# Last subplot: Q-Q plot for Sales
ax_qq = fig.add_subplot(gs[1, 2])
stats.probplot(df['Sales'], dist="norm", plot=ax_qq)
ax_qq.set_title('Q-Q Plot: Sales\n(Normality Check)', fontweight='bold', fontsize=11)
ax_qq.get_lines()[0].set(color='#3498db', markersize=3, alpha=0.5)
ax_qq.get_lines()[1].set(color='red', linewidth=2)

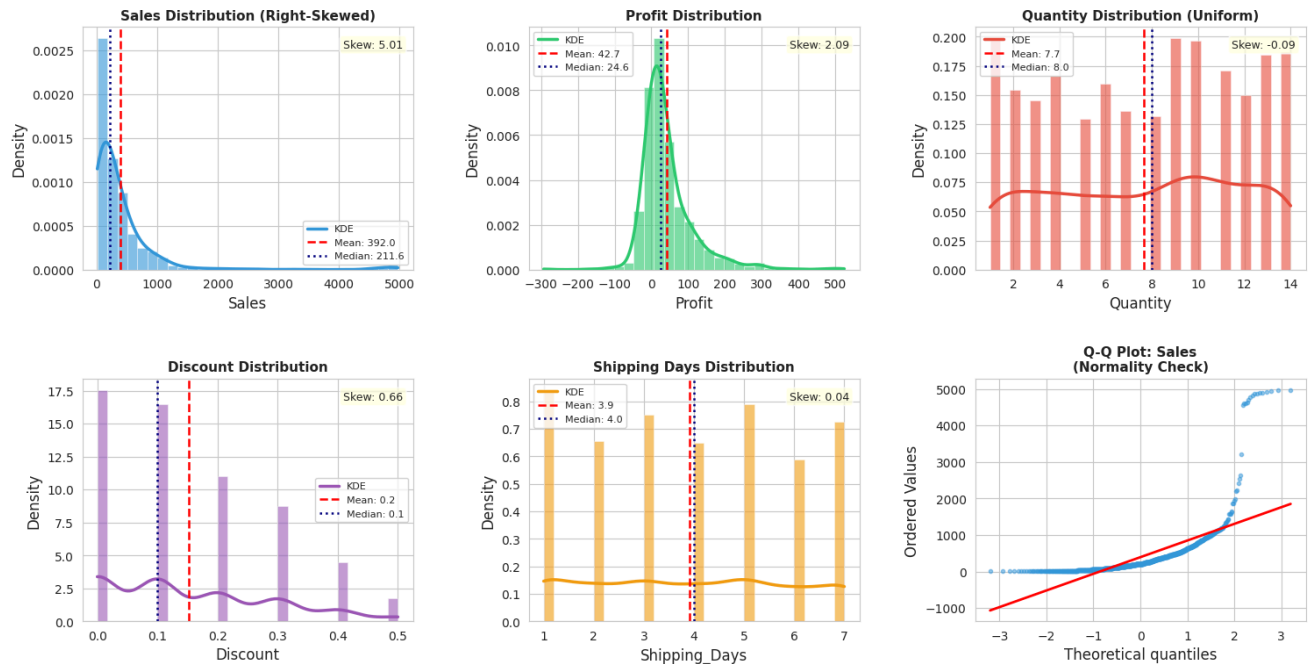
plt.savefig('distributions.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n📊 Distribution Insights:")
for col in num_cols:
    skew = df[col].skew()
    if abs(skew) < 0.5: skew_type = "approximately symmetric"
    elif skew > 0:      : skew_type = "right-skewed (positively skewed)"
    else:               : skew_type = "left-skewed (negatively skewed)"
    print(f"    {col:<15} → Skewness: {skew:+.2f} → {skew_type}")

```

SECTION 6: DISTRIBUTION ANALYSIS

Distribution Analysis of Numerical Variables



📊 Distribution Insights:

Sales	→ Skewness: +5.01 → right-skewed (positively skewed)
Profit	→ Skewness: +2.09 → right-skewed (positively skewed)
Quantity	→ Skewness: -0.09 → approximately symmetric
Discount	→ Skewness: +0.66 → right-skewed (positively skewed)
Shipping_Days	→ Skewness: +0.04 → approximately symmetric


```

# -----
# SECTION 7: Outlier Detection & Analysis
# -----

print("\n" + "=" * 60)
print("    SECTION 7: OUTLIER DETECTION & ANALYSIS")
print("=" * 60)

def detect_outliers_iqr(data, col):
    """Detect outliers using IQR method."""
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]
    return outliers, lower_bound, upper_bound, Q1, Q3, IQR

def detect_outliers_zscore(data, col, threshold=3):
    """Detect outliers using Z-Score method."""
    z_scores = np.abs(stats.zscore(data[col].dropna()))
    return np.where(z_scores > threshold)[0]

print("\n🚀 IQR-Based Outlier Detection:")
print("-" * 70)
print(f"{'Column':<15} {'Q1':>8} {'Q3':>8} {'IQR':>8} "
      f"{'Lower':>10} {'Upper':>10} {'Outliers':>10} {'%':>6}")
print("-" * 70)

outlier_summary = {}
for col in ['Sales', 'Profit', 'Quantity', 'Discount']:
    outliers, lb, ub, q1, q3, iqr = detect_outliers_iqr(df, col)
    pct = len(outliers) / len(df) * 100
    outlier_summary[col] = {
        'outliers': outliers, 'lower': lb, 'upper': ub,
        'count': len(outliers), 'pct': pct
    }
    print(f"{'col':<15} {'q1':>8.2f} {'q3':>8.2f} {'iqr':>8.2f} "
          f"{'lb':>10.2f} {'ub':>10.2f} {'len(outliers):>10} {'pct:>5.1f}%")

# Z-Score comparison
print("\n🚀 Z-Score Outlier Detection (threshold = 3σ):")
for col in ['Sales', 'Profit']:
    z_idx = detect_outliers_zscore(df, col)
    print(f"    {col}: {len(z_idx)} outliers detected")

# — Box Plots Visualization —
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Outlier Detection – Box Plots', fontsize=16,
             fontweight='bold', y=1.01)

cols_to_plot = ['Sales', 'Profit', 'Quantity', 'Discount']
colors_box = ['#3498db', '#2ecc71', '#e74c3c', '#9b59b6']

for ax, col, color in zip(axes.flatten(), cols_to_plot, colors_box):
    # Box plot
    bp = ax.boxplot(df[col].dropna(), vert=True, patch_artist=True,
                    notch=True, whis=1.5,
                    boxprops=dict(facecolor=color, alpha=0.6),
                    medianprops=dict(color='black', linewidth=2),
                    whiskerprops=dict(linewidth=1.5),
                    flierprops=dict(marker='o', markerfacecolor='red',
                                    markersize=5, alpha=0.6))

    # Overlay scatter for visibility
    y = df[col].dropna()
    x = np.random.normal(1, 0.04, size=len(y))
    ax.scatter(x, y, alpha=0.2, s=8, color=color)

    # Outlier bounds
    lb = outlier_summary[col]['lower']
    ub = outlier_summary[col]['upper']
    ax.axhline(ub, color='red', linestyle='--', linewidth=1,
               label=f'Upper: {ub:.1f}')
    ax.axhline(lb, color='blue', linestyle='--', linewidth=1,
               label=f'Lower: {lb:.1f}')

```

```

count = outlier_summary[col]['count']
pct = outlier_summary[col]['pct']
ax.set_title(f'{col}\n({count} outliers = {pct:.1f}%)',
             fontweight='bold')
ax.set_ylabel(col)
ax.legend(fontsize=9)
ax.set_xticks([])

plt.tight_layout()
plt.savefig('outliers.png', dpi=150, bbox_inches='tight')
plt.show()

# Violin plots for richer view
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Violin Plots – Sales & Profit by Category',
             fontsize=15, fontweight='bold')

sns.violinplot(data=df, x='Category', y='Sales',
               palette='Blues', inner='quartile', ax=axes[0])
axes[0].set_title('Sales Distribution by Category', fontweight='bold')
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Sales ($)')

sns.violinplot(data=df, x='Category', y='Profit',
               palette='Greens', inner='quartile', ax=axes[1])
axes[1].set_title('Profit Distribution by Category', fontweight='bold')
axes[1].set_xlabel('Category')
axes[1].set_ylabel('Profit ($)')

plt.tight_layout()
plt.savefig('violin_plots.png', dpi=150, bbox_inches='tight')
plt.show()

```


SECTION 7: OUTLIER DETECTION & ANALYSIS

🔪 IQR-Based Outlier Detection:

Column	Q1	Q3	IQR	Lower	Upper	Outliers	%
--------	----	----	-----	-------	-------	----------	---

```
# -----
# SECTION 8: Categorical Variable Analysis
# -----

print("\n" + "=" * 60)
print("    SECTION 8: CATEGORICAL VARIABLE ANALYSIS")
print("=" * 60)

cat_cols = ['Category', 'Sub_Category', 'Region', 'Segment', 'Ship_Mode']

for col in cat_cols:
    vc = df[col].value_counts()
    print(f"\n🔪 {col} Distribution:")
    for val, cnt in vc.items():
        pct = cnt / len(df) * 100
        bar = '█' * int(pct / 2)
        print(f"    {val:<22} {cnt:>5} ({pct:>5.1f}%) {bar}")

# --- Visualization ---
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Categorical Variable Distributions',
             fontsize=17, fontweight='bold')

palette_sets = [
    sns.color_palette("Set2", 3),
    sns.color_palette("Paired", 12),
    sns.color_palette("husl", 4),
    sns.color_palette("Set3", 3),
    sns.color_palette("pastel", 4),
]

# Plot 1: Category - Pie Chart
ax = axes[0, 0]
cat_counts = df['Category'].value_counts()
wedge_props = dict(width=0.6, edgecolor='white', linewidth=2)
ax.pie(cat_counts.values,
       labels=cat_counts.index,
       autopct='%1.1f%%',
       colors=palette_sets[0],
       wedgeprops=wedge_props,
       startangle=90)
ax.set_title('Category Distribution\n(Donut Chart)', fontweight='bold')

# Plot 2: Region - Horizontal Bar
ax = axes[0, 1]
region_counts = df['Region'].value_counts()
bars = ax.barh(region_counts.index,
               region_counts.values,
               color=palette_sets[2],
               edgecolor='white', linewidth=1.5)
ax.set_title('Orders by Region', fontweight='bold')
ax.set_xlabel('Number of Orders')
for bar, val in zip(bars, region_counts.values):
    ax.text(bar.get_width() + 3, bar.get_y() + bar.get_height() / 2,
           f'{val} ({val/len(df)*100:.1f}%)',
           va='center', fontsize=9)
ax.set_xlim(0, max(region_counts.values) * 1.25)

# Plot 3: Segment - Bar Chart
ax = axes[0, 2]
seg_counts = df['Segment'].value_counts()
bars = ax.bar(seg_counts.index, seg_counts.values,
               color=palette_sets[3],
               edgecolor='white', linewidth=1.5)
ax.set_title('Orders by Customer Segment', fontweight='bold')
ax.set_ylabel('Count')
for bar, val in zip(bars, seg_counts.values):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 3,
           f'{val} ({val/len(df)*100:.1f}%)')
```

```

        f'{val}/n({val}/len(df)*100.1f)%',
        ha='center', va='bottom', fontsize=9)

# Plot 4: Sub-Category - Sorted Bar
ax = axes[1, 0]
sub_counts = df['Sub_Category'].value_counts()
colors_sub = sns.color_palette("tab20", len(sub_counts))
bars = ax.bar(sub_counts.index, sub_counts.values,
              color=colors_sub, edgecolor='white')
ax.set_title('Orders by Sub-Category', fontweight='bold')
ax.set_ylabel('Count')
ax.set_xticklabels(sub_counts.index, rotation=45, ha='right')
for bar, val in zip(bars, sub_counts.values):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1,
            f'{val}', ha='center', va='bottom', fontsize=8)

# Plot 5: Ship Mode - Pie
ax = axes[1, 1]
ship_counts = df['Ship_Mode'].value_counts()
ax.pie(ship_counts.values,
       labels=ship_counts.index,
       autopct='%1.1f%%',
       colors=palette_sets[4],
       startangle=140,
       shadow=True)
ax.set_title('Shipping Mode Preference', fontweight='bold')

# Plot 6: Category x Segment Heatmap
ax = axes[1, 2]
cross_tab = pd.crosstab(df['Category'], df['Segment'])
sns.heatmap(cross_tab,
            annot=True, fmt='d',
            cmap='YlOrRd',
            ax=ax,
            linewidths=0.5,
            cbar_kws={'label': 'Count'})
ax.set_title('Category x Segment Cross-Tab', fontweight='bold')
ax.set_xlabel('Segment')
ax.set_ylabel('Category')

plt.tight_layout(pad=2)
plt.savefig('categorical_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

```

SECTION 8: CATEGORICAL VARIABLE ANALYSIS

Category Distribution:

Technology	365	(36.5%)	
Office Supplies	329	(32.9%)	
Furniture	306	(30.6%)	

Sub_Category Distribution:

Computers	97	(9.7%)	
Storage	95	(9.5%)	
Phones	95	(9.5%)	
Copiers	90	(9.0%)	
Chairs	86	(8.6%)	
Accessories	83	(8.3%)	
Paper	83	(8.3%)	
Furnishings	80	(8.0%)	
Binders	80	(8.0%)	
Tables	78	(7.8%)	
Appliances	71	(7.1%)	
Bookcases	62	(6.2%)	

Region Distribution:

East	253	(25.3%)	
West	250	(25.0%)	
South	250	(25.0%)	
North	247	(24.7%)	

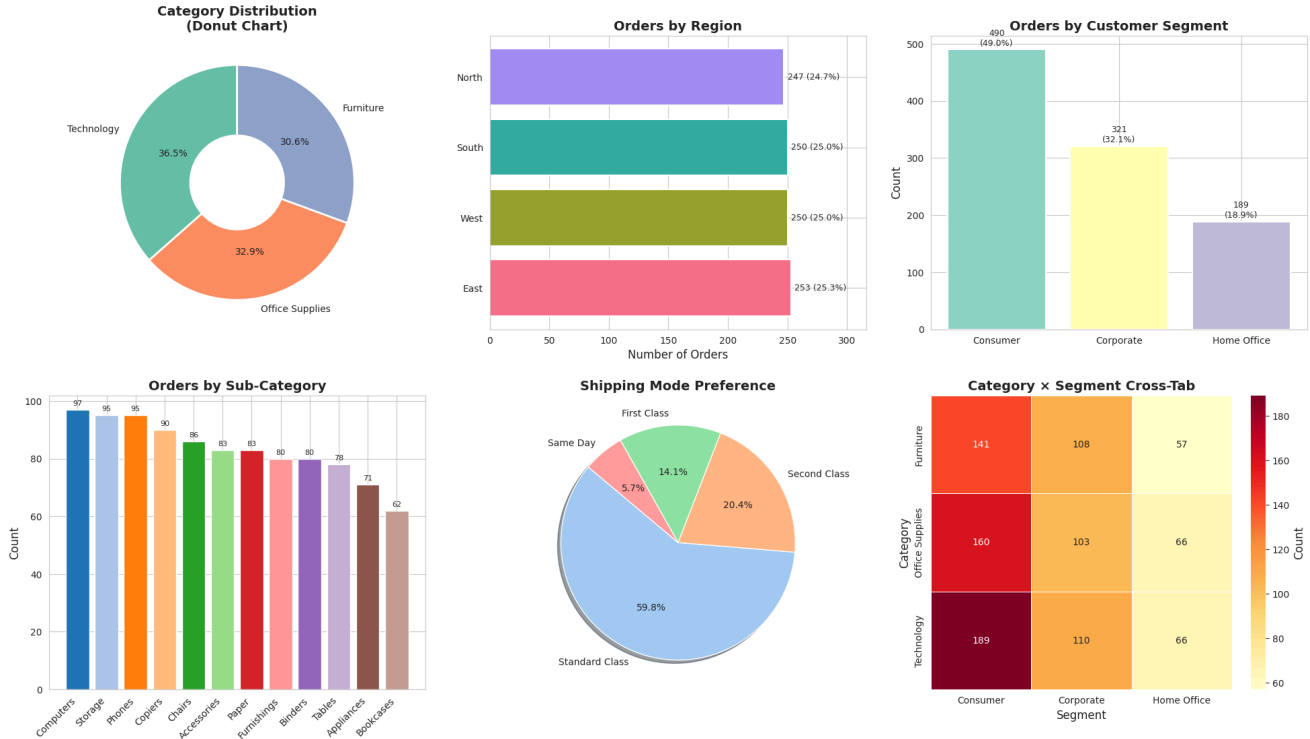
Segment Distribution:

Consumer	490	(49.0%)	
Corporate	321	(32.1%)	
Home Office	189	(18.9%)	

Ship_Mode Distribution:

Standard Class	598	(59.8%)	
Second Class	204	(20.4%)	
First Class	141	(14.1%)	
Same Day	57	(5.7%)	

Categorical Variable Distributions



```

#
print("\n" + "=" * 60)
print("    SECTION 9: COMPARATIVE ANALYSIS")
print("=" * 60)

# 9.1 Sales by Category
print("\n🔥 9.1 – Average Sales by Category:")
cat_sales = df.groupby('Category')['Sales'].agg(
    ['mean', 'median', 'sum', 'count', 'std']
).round(2)
cat_sales.columns = ['Mean', 'Median', 'Total Sales', 'Count', 'Std Dev']
cat_sales['Revenue Share %'] = (cat_sales['Total Sales'] /
                                cat_sales['Total Sales'].sum() * 100).round(1)

print(cat_sales.to_string())

# 9.2 Profit by Region
print("\n🔥 9.2 – Profit Metrics by Region:")
reg_profit = df.groupby('Region')['Profit'].agg(
    ['mean', 'median', 'sum']
).round(2)
reg_profit.columns = ['Avg Profit', 'Median Profit', 'Total Profit']
print(reg_profit.to_string())

# 9.3 Discount impact on profit
print("\n🔥 9.3 – Discount Impact Analysis:")
df['Discount_Bin'] = pd.cut(df['Discount'],
                            bins=[-0.01, 0.0, 0.1, 0.2, 0.3, 0.5],
                            labels=['0%', '1-10%', '11-20%', '21-30%', '>30%'])
disc_analysis = df.groupby('Discount_Bin', observed=True).agg(
    Count=('Sales', 'count'),
    Avg_Sales=('Sales', 'mean'),
    Avg_Profit=('Profit', 'mean'),
    Total_Profit=('Profit', 'sum')
).round(2)
print(disc_analysis.to_string())

# — Visualizations —
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Comparative Analysis: Sales, Profit & Discount Insights',
             fontsize=16, fontweight='bold')

# Plot 1: Mean Sales by Category
ax = axes[0, 0]
cat_means = df.groupby('Category')['Sales'].mean().sort_values(ascending=False)
bars = ax.bar(cat_means.index, cat_means.values,
              color=['#3498db', '#e74c3c', '#2ecc71'],
              edgecolor='white', linewidth=1.5)
ax.set_title('Average Sales by Category', fontweight='bold')
ax.set_ylabel('Average Sales ($)')
for bar, val in zip(bars, cat_means.values):
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 5,
            f'${val:.0f}', ha='center', fontsize=10, fontweight='bold')

# Plot 2: Total Profit by Region
ax = axes[0, 1]
reg_totals = df.groupby('Region')['Profit'].sum().sort_values(ascending=True)
colors_reg = ['#e74c3c' if v < 0 else '#2ecc71' for v in reg_totals]
bars = ax.barh(reg_totals.index, reg_totals.values,
               color=colors_reg, edgecolor='white')
ax.set_title('Total Profit by Region', fontweight='bold')
ax.set_xlabel('Total Profit ($)')
ax.axvline(0, color='black', linewidth=1)
for bar, val in zip(bars, reg_totals.values):
    xpos = bar.get_width() + 50 if val > 0 else bar.get_width() - 200
    ax.text(xpos, bar.get_y() + bar.get_height() / 2,
            f'${val:,.0f}', va='center', fontsize=9)

# Plot 3: Sales by Segment – Grouped
ax = axes[0, 2]
seg_cat = df.groupby(['Segment', 'Category'])['Sales'].mean().unstack()
seg_cat.plot(kind='bar', ax=ax, colormap='Set2',
             edgecolor='white', linewidth=1)
ax.set_title('Avg Sales: Segment x Category', fontweight='bold')
ax.set_xlabel('Customer Segment')
ax.set_ylabel('Average Sales ($)')
ax.legend(title='Category', fontsize=8)

```

```

ax.set_xticklabels(ax.get_xticklabels(), rotation=0)

# Plot 4: Discount vs Profit (Scatter)
ax = axes[1, 0]
for cat, color in zip(df['Category'].unique(), ['#3498db', '#e74c3c', '#2ecc71']):
    subset = df[df['Category'] == cat]
    ax.scatter(subset['Discount'], subset['Profit'],
               alpha=0.4, s=30, color=color, label=cat)

# Trend line
z = np.polyfit(df['Discount'], df['Profit'], 1)
p = np.poly1d(z)
x_line = np.linspace(0, 0.5, 100)
ax.plot(x_line, p(x_line), 'r--', linewidth=2, label=f'Trend')
ax.axhline(0, color='black', linewidth=1, linestyle=':')
ax.set_title('Discount vs Profit (Scatter)', fontweight='bold')
ax.set_xlabel('Discount Rate')
ax.set_ylabel('Profit ($)')
ax.legend(fontsize=8)

corr_dp = df['Discount'].corr(df['Profit'])
ax.text(0.02, 0.95, f'r = {corr_dp:.3f}',
       transform=ax.transAxes,
       bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8),
       fontsize=10)

# Plot 5: Avg Profit by Discount Bin
ax = axes[1, 1]
disc_pivot = disc_analysis['Avg_Profit']
bar_colors = ['#2ecc71' if v > 0 else '#e74c3c' for v in disc_pivot]
bars = ax.bar(disc_pivot.index.astype(str), disc_pivot.values,
              color=bar_colors, edgecolor='white', linewidth=1.5)
ax.axhline(0, color='black', linewidth=1)
ax.set_title('Average Profit by Discount Range', fontweight='bold')
ax.set_xlabel('Discount Range')
ax.set_ylabel('Average Profit ($)')
for bar, val in zip(bars, disc_pivot.values):
    ypos = bar.get_height() + 1 if val > 0 else bar.get_height() - 4
    ax.text(bar.get_x() + bar.get_width() / 2, ypos,
            f'${val:.1f}', ha='center', fontsize=9, fontweight='bold')

# Plot 6: Sales by Ship Mode
ax = axes[1, 2]
ship_stats = df.groupby('Ship_Mode')['Sales'].agg(['mean', 'std']).reset_index()
ship_stats.columns = ['Ship_Mode', 'Mean', 'Std']
ship_stats = ship_stats.sort_values('Mean', ascending=False)

bars = ax.bar(ship_stats['Ship_Mode'], ship_stats['Mean'],
              yerr=ship_stats['Std'],
              color=sns.color_palette("pastel", 4),
              edgecolor='white', capsize=5, linewidth=1.5)
ax.set_title('Average Sales by Shipping Mode\n(with Std Dev error bars)',
             fontweight='bold')
ax.set_xlabel('Shipping Mode')
ax.set_ylabel('Average Sales ($)')
ax.set_xticklabels(ship_stats['Ship_Mode'], rotation=15, ha='right')

plt.tight_layout(pad=2)
plt.savefig('comparative_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

```


SECTION 9: COMPARATIVE ANALYSIS

9.1 – Average Sales by Category:

	Mean	Median	Total Sales	Count	Std Dev	Revenue Share %
Category						
Furniture	365.71	238.04	111907.62	306	524.13	28.5
Office Supplies	249.71	116.89	82153.27	329	690.73	21.0
Technology	542.24	389.02	197915.98	365	682.05	50.5

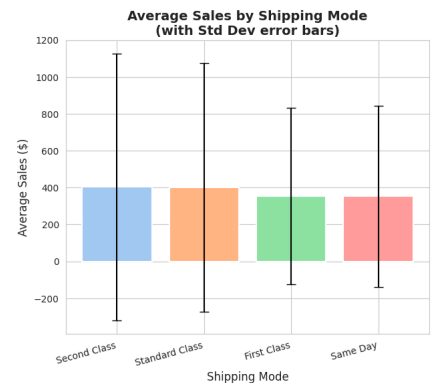
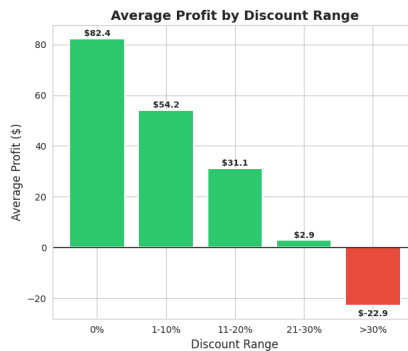
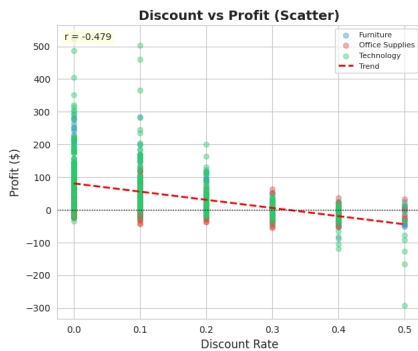
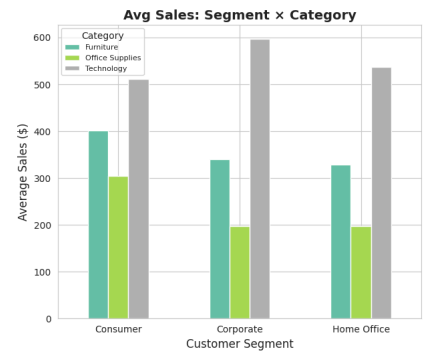
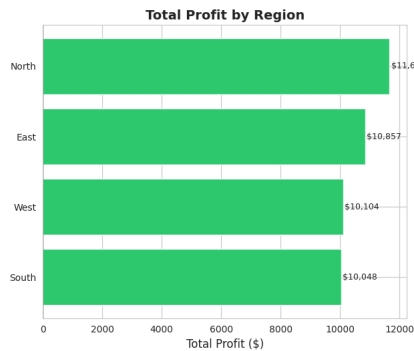
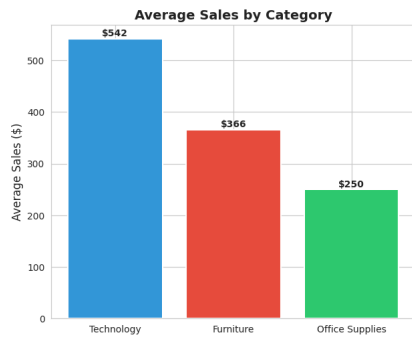
9.2 – Profit Metrics by Region:

	Avg Profit	Median Profit	Total Profit
Region			
East	42.91	27.94	10857.10
North	47.25	23.78	11670.39
South	40.19	21.85	10048.25
West	40.42	24.64	10104.06

9.3 – Discount Impact Analysis:

	Count	Avg_Sales	Avg_Profit	Total_Profit
Discount_Bin				
0%	292	392.67	82.38	24056.41
1-10%	274	438.13	54.21	14854.54
11-20%	184	397.21	31.15	5731.00
21-30%	146	351.73	2.89	421.97
>30%	104	315.69	-22.92	-2384.12

Comparative Analysis: Sales, Profit & Discount Insights



```
#  
# SECTION 10: Correlation Analysis  
#  
  
print("\n" + "=" * 60)  
print("  SECTION 10: CORRELATION ANALYSIS")  
print("=" * 60)  
  
# Compute correlation matrices  
corr_pearson = df[num_cols].corr(method='pearson')  
corr_spearman = df[num_cols].corr(method='spearman')  
  
print("\n🚀 Pearson Correlation Matrix:")  
print(corr_pearson.round(2).to_string())
```

```

print(corr_pearson.round(3).to_string())
print("\n🔥 Spearman Correlation Matrix:")
print(corr_spearman.round(3).to_string())

# — Heatmaps —————
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('Correlation Analysis', fontsize=16, fontweight='bold')

mask = np.triu(np.ones_like(corr_pearson, dtype=bool))

# Pearson Heatmap
sns.heatmap(corr_pearson,
            mask=mask,
            annot=True,
            fmt='.3f',
            cmap='RdBu_r',
            center=0,
            vmin=-1, vmax=1,
            square=True,
            linewidths=0.8,
            cbar_kws={'shrink': 0.8, 'label': 'Correlation'},
            ax=axes[0])
axes[0].set_title('Pearson Correlation Heatmap\n(Lower Triangle)',
                  fontweight='bold')

# Spearman Heatmap (full)
sns.heatmap(corr_spearman,
            annot=True,
            fmt='.3f',
            cmap='coolwarm',
            center=0,
            vmin=-1, vmax=1,
            square=True,
            linewidths=0.8,
            cbar_kws={'shrink': 0.8, 'label': 'Correlation'},
            ax=axes[1])
axes[1].set_title('Spearman Correlation Heatmap\n(Full Matrix)',
                  fontweight='bold')

plt.tight_layout()
plt.savefig('correlation_heatmaps.png', dpi=150, bbox_inches='tight')
plt.show()

# Key correlation pairs
print("\n🔥 Top Correlation Pairs:")
corr_pairs = corr_pearson.unstack().sort_values(key=abs, ascending=False)
corr_pairs = corr_pairs[corr_pairs != 1.0].drop_duplicates()
for (c1, c2), val in corr_pairs.head(8).items():
    strength = ("strong" if abs(val) > 0.6
               else "moderate" if abs(val) > 0.3
               else "weak")
    direction = "positive" if val > 0 else "negative"
    print(f"    {c1} ↔ {c2}: r={val:.3f} → {strength} {direction} correlation")

```

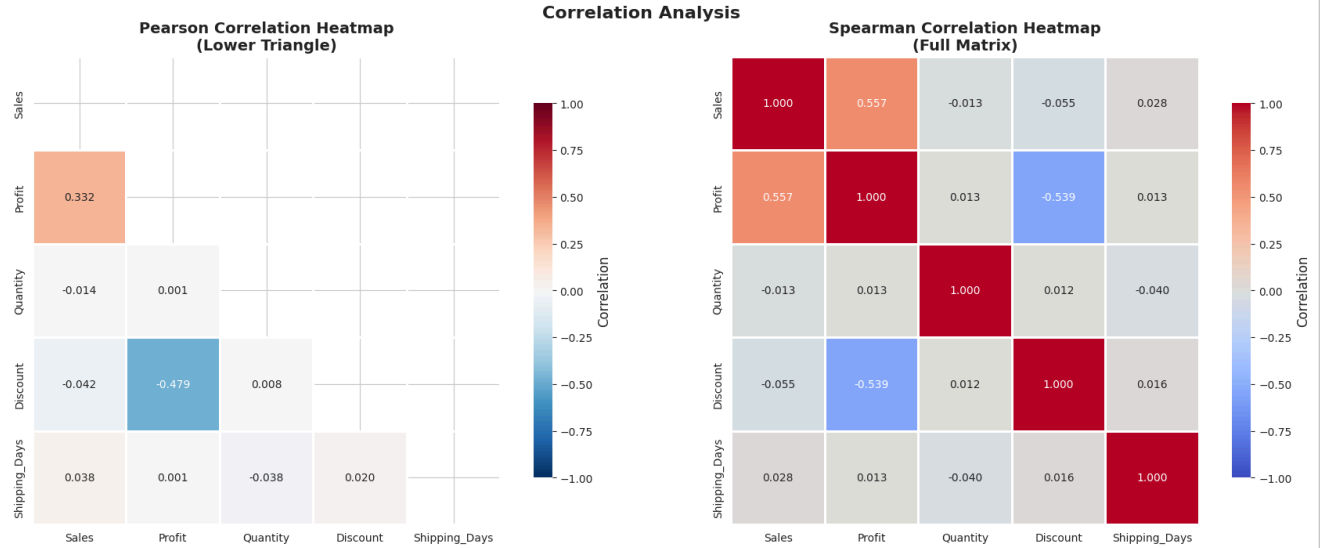
SECTION 10: CORRELATION ANALYSIS

Pearson Correlation Matrix:

	Sales	Profit	Quantity	Discount	Shipping_Days
Sales	1.000	0.332	-0.014	-0.042	0.038
Profit	0.332	1.000	0.001	-0.479	0.001
Quantity	-0.014	0.001	1.000	0.008	-0.038
Discount	-0.042	-0.479	0.008	1.000	0.020
Shipping_Days	0.038	0.001	-0.038	0.020	1.000

Spearman Correlation Matrix:

	Sales	Profit	Quantity	Discount	Shipping_Days
Sales	1.000	0.557	-0.013	-0.055	0.028
Profit	0.557	1.000	0.013	-0.539	0.013
Quantity	-0.013	0.013	1.000	0.012	-0.040
Discount	-0.055	-0.539	0.012	1.000	0.016
Shipping_Days	0.028	0.013	-0.040	0.016	1.000



Top Correlation Pairs:

Discount ↔ Profit: $r = -0.479$ → moderate negative correlation
Sales ↔ Profit: $r = 0.332$ → moderate positive correlation
Discount ↔ Sales: $r = -0.042$ → weak negative correlation
Quantity ↔ Shipping_Days: $r = -0.038$ → weak negative correlation
Sales ↔ Shipping_Days: $r = 0.038$ → weak positive correlation
Shipping_Days ↔ Discount: $r = 0.020$ → weak positive correlation
Quantity ↔ Sales: $r = -0.014$ → weak negative correlation
Discount ↔ Quantity: $r = 0.008$ → weak positive correlation

```
#  
# SECTION 11: Time Series Analysis  
#  
  
print("\n" + "=" * 60)  
print("  SECTION 11: TIME SERIES ANALYSIS")  
print("=" * 60)  
  
# Monthly aggregation  
monthly = (df.groupby(['Year', 'Month', 'Month_Name'])  
           .agg(Total_Sales=('Sales', 'sum'),  
               Total_Profit=('Profit', 'sum'),  
               Order_Count=('Sales', 'count'))  
           .reset_index()  
           .sort_values(['Year', 'Month']))  
  
monthly['Period'] = monthly['Month_Name'] + ' ' + monthly['Year'].astype(str)  
  
# Yearly comparison  
yearly = df.groupby('Year').agg(  
    Total_Sales=('Sales', 'sum'),  
    Total_Profit=('Profit', 'sum'),
```

```

    Avg_Sales=('Sales', 'mean'),
    Order_Count=('Sales', 'count')
).round(2)
yearly['Profit_Margin%'] = (yearly['Total_Profit'] /
                           yearly['Total_Sales'] * 100).round(2)

print("\n🔥 Yearly Performance Summary:")
print(yearly.to_string())

print("\n🔥 Best & Worst Months (by Total Sales):")
best_month = monthly.nlargest(3, 'Total_Sales')[['Period', 'Total_Sales', 'Total_Profit']]
worst_month = monthly.nsmallest(3, 'Total_Sales')[['Period', 'Total_Sales', 'Total_Profit']]
print(f"    Top 3 months:\n{best_month.to_string(index=False)}")
print(f"    Bottom 3 months:\n{worst_month.to_string(index=False)}")

# — Visualization —————
fig = plt.figure(figsize=(20, 15))
fig.suptitle('Time Series Analysis – Sales & Profit Trends',
             fontsize=17, fontweight='bold')
gs = gridspec.GridSpec(3, 2, hspace=0.45, wspace=0.3)

# Plot 1: Monthly Sales Trend
ax1 = fig.add_subplot(gs[0, :])
x = range(len(monthly))
ax1.fill_between(x, monthly['Total_Sales'], alpha=0.3, color='#3498db')
ax1.plot(x, monthly['Total_Sales'], color='#3498db',
         linewidth=2.5, marker='o', markersize=4, label='Total Sales')
ax1.plot(x, monthly['Total_Profit'], color='#2ecc71',
         linewidth=2, marker='s', markersize=4, label='Total Profit')

# Year markers
for yr in [2021, 2022, 2023]:
    yr_idx = monthly[monthly['Year'] == yr].index[0] - monthly.index[0]
    ax1.axvline(yr_idx, color='red', linestyle='--',
                linewidth=1.5, alpha=0.7, label=f'Year {yr}')

ax1.set_title('Monthly Sales & Profit Trend (2021–2023)', fontweight='bold')
ax1.set_xticks(x[::3])
ax1.set_xticklabels(monthly['Period'].iloc[::3], rotation=45, ha='right', fontsize=8)
ax1.set_ylabel('Amount ($)')
ax1.legend(fontsize=9)
ax1.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda val, _: f'${val:,.0f}'))

# Plot 2: Yearly Comparison Bar
ax2 = fig.add_subplot(gs[1, 0])
years = yearly.index
x_pos = np.arange(len(years))
width = 0.35
bars1 = ax2.bar(x_pos - width/2, yearly['Total_Sales'],
                 width, label='Total Sales', color='#3498db',
                 edgecolor='white', linewidth=1.5)
bars2 = ax2.bar(x_pos + width/2, yearly['Total_Profit'],
                 width, label='Total Profit', color='#2ecc71',
                 edgecolor='white', linewidth=1.5)
ax2.set_title('Year-over-Year Comparison', fontweight='bold')
ax2.set_xticks(x_pos)
ax2.set_xticklabels(years)
ax2.set_ylabel('Amount ($)')
ax2.legend()
ax2.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda val, _: f'${val:,.0f}'))

# Plot 3: Monthly Heatmap
ax3 = fig.add_subplot(gs[1, 1])
pivot_heat = monthly.pivot_table(values='Total_Sales',
                                  index='Month', columns='Year',
                                  aggfunc='sum')
sns.heatmap(pivot_heat, annot=True, fmt='%.0f',
            cmap='YlOrRd', ax=ax3,
            linewidths=0.5, cbar_kws={'label': 'Total Sales ($)'})
ax3.set_title('Monthly Sales Heatmap\n(Month x Year)', fontweight='bold')
ax3.set_xlabel('Year')
ax3.set_ylabel('Month')

# Plot 4: Category sales over time
ax4 = fig.add_subplot(gs[2, 0])
for cat, color in zip(df['Category'].unique(),

```

```

        ['#3498db', '#e74c3c', '#2ecc71']):
cat_monthly = (df[df['Category'] == cat]
               .groupby(['Year', 'Month'])['Sales']
               .sum()
               .reset_index())
cat_monthly['Period_Num'] = (cat_monthly['Year'] - 2021) * 12 + cat_monthly['Month']
ax4.plot(cat_monthly['Period_Num'], cat_monthly['Sales'],
         label=cat, color=color, linewidth=2, alpha=0.8)

ax4.set_title('Monthly Sales Trend by Category', fontweight='bold')
ax4.set_xlabel('Period')
ax4.set_ylabel('Total Sales ($)')
ax4.legend(fontsize=9)

# Plot 5: Order volume by month
ax5 = fig.add_subplot(gs[2, 1])
month_order = df['Month_Name'].value_counts().reindex(
    ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
     'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
ax5.bar(month_order.index, month_order.values,
        color=plt.cm.RdYlGn(np.linspace(0.2, 0.9, 12)),
        edgecolor='white')
ax5.set_title('Order Volume by Month\n(All Years Combined)',
             fontweight='bold')
ax5.set_xlabel('Month')
ax5.set_ylabel('Number of Orders')

plt.savefig('time_series.png', dpi=150, bbox_inches='tight')
plt.show()

```

SECTION 11: TIME SERIES ANALYSIS

📌 Yearly Performance Summary:

	Total_Sales	Total_Profit	Avg_Sales	Order_Count	Profit_Margin%
Year					
2021	140125.57	14886.46	419.54	334	10.62
2022	127907.06	14498.75	384.11	333	11.34
2023	123944.24	13294.60	372.20	333	10.73

📌 Best & Worst Months (by Total Sales):

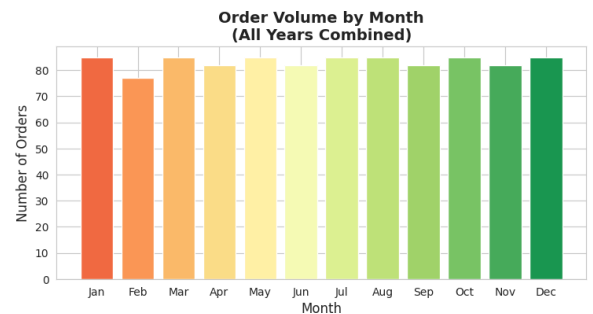
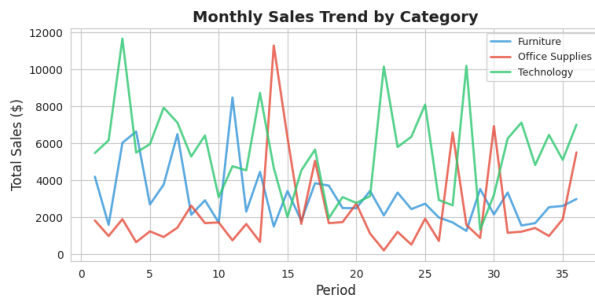
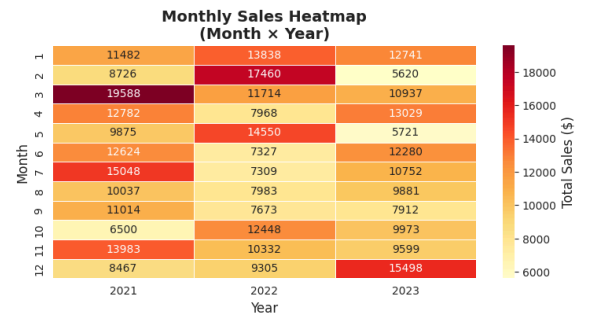
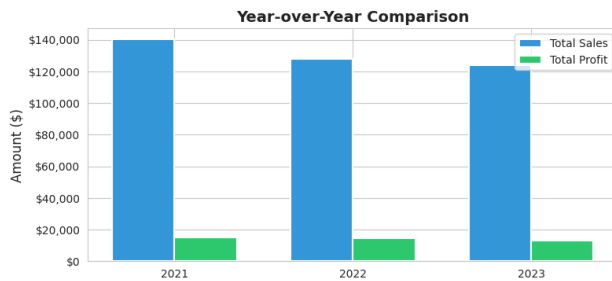
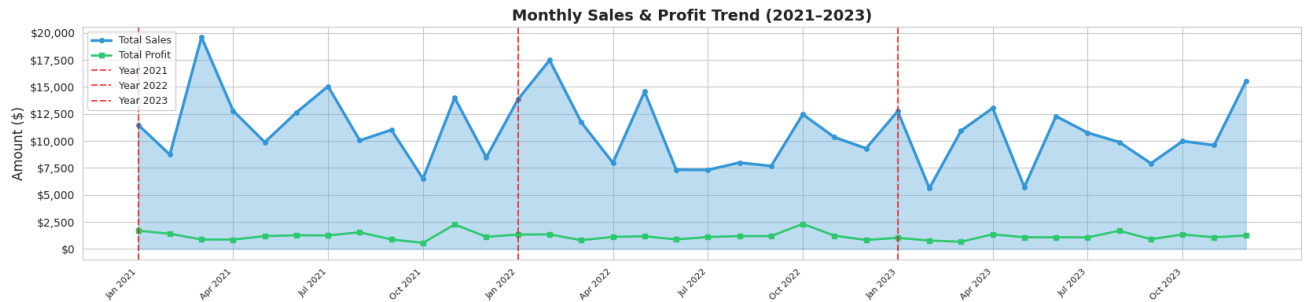
Top 3 months:

Period	Total_Sales	Total_Profit
Mar 2021	19588.097894	871.214613
Feb 2022	17459.529421	1347.166651
Dec 2023	15498.460633	1248.334871

Bottom 3 months:

Period	Total_Sales	Total_Profit
Feb 2023	5620.267867	779.442135
May 2023	5720.714596	1080.839166
Oct 2021	6499.987043	558.284742

Time Series Analysis — Sales & Profit Trends



```

print("\n" + "=" * 60)
print("    SECTION 12: MULTI-VARIATE ANALYSIS")
print("=" * 60)

# 12.1 Pair Plot
fig = plt.figure(figsize=(14, 12))

pair_data = df[['Sales', 'Profit', 'Quantity',
                'Discount', 'Category']].dropna()

pair_grid = sns.pairplot(
    pair_data,
    hue='Category',
    diag_kind='kde',
    plot_kws={'alpha': 0.4, 's': 25},
    palette={'Technology': '#3498db',
            'Furniture': '#e74c3c',
            'Office Supplies': '#2ecc71'}
)
pair_grid.figure.suptitle('Pair Plot: Numerical Variables by Category',
                          y=1.02, fontsize=15, fontweight='bold')

plt.savefig('pairplot.png', dpi=120, bbox_inches='tight')
plt.show()
print("✅ Pair plot saved!")

# 12.2 Bubble Chart
fig, ax = plt.subplots(figsize=(14, 8))

for cat, color, marker in zip(
    df['Category'].unique(),
    ['#3498db', '#e74c3c', '#2ecc71'],
    ['o', 's', '^']):
    subset = df[df['Category'] == cat].sample(100, random_state=42)
    scatter = ax.scatter(
        subset['Sales'],
        subset['Profit'],
        s=subset['Quantity'] * 20,
        alpha=0.5,
        color=color,
        label=cat,
        marker=marker,
        edgecolors='white',
        linewidths=0.5
    )

ax.axhline(0, color='black', linewidth=1, linestyle='--', alpha=0.5)
ax.set_title('Bubble Chart: Sales vs Profit\n'
             '(Bubble Size = Quantity)', fontsize=14, fontweight='bold')
ax.set_xlabel('Sales ($)', fontsize=12)
ax.set_ylabel('Profit ($)', fontsize=12)
ax.legend(title='Category', fontsize=10)

# Quadrant labels
ax_xlim = ax.get_xlim()
ax_ylim = ax.get_ylim()
mid_x = (ax_xlim[0] + ax_xlim[1]) / 2
mid_y = 0

quads = [
    ('High Sales\nLow Profit', mid_x * 1.3, -30, '#e74c3c'),
    ('High Sales\nHigh Profit', mid_x * 1.3, 80, '#2ecc71'),
    ('Low Sales\nHigh Profit', mid_x * 0.3, 80, '#3498db'),
    ('Low Sales\nLow Profit', mid_x * 0.3, -30, '#e67e22'),
]

for label, x, y, c in quads:
    ax.text(x, y, label, fontsize=8, color=c, alpha=0.7,
            ha='center', fontstyle='italic')

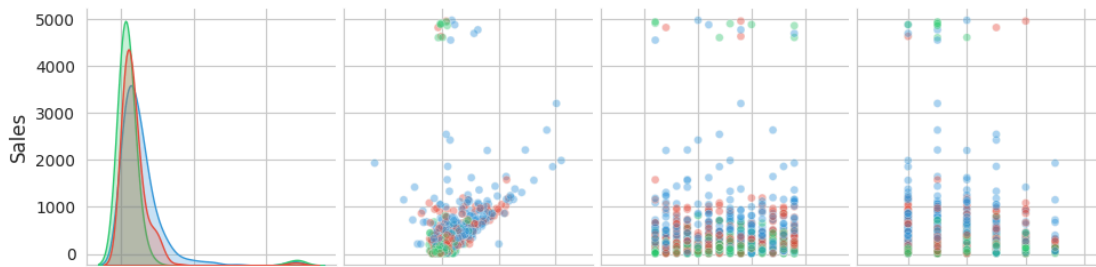
plt.tight_layout()
plt.savefig('bubble_chart.png', dpi=150, bbox_inches='tight')
plt.show()

```


SECTION 12: MULTI-VARIATE ANALYSIS

<Figure size 1400x1200 with 0 Axes>

Pair Plot: Numerical Variables by Category



```
# -----
# SECTION 13: Comprehensive Summary Dashboard
# -----

print("\n" + "=" * 60)
print("    SECTION 13: EDA SUMMARY DASHBOARD")
print("=" * 60)

fig = plt.figure(figsize=(22, 16))
fig.patch.set_facecolor('#f8f9fa')
fig.suptitle('🌐 Global Superstore – EDA Summary Dashboard',
             fontsize=20, fontweight='bold', y=0.98, color='#2c3e50')

gs = gridspec.GridSpec(3, 4, figure=fig,
                       hspace=0.55, wspace=0.4,
                       top=0.93, bottom=0.06,
                       left=0.06, right=0.97)

# — KPI Cards (text panels) —————
kpis = [
    ('Total Revenue', f"${df['Sales'].sum():.0f}", '#3498db'),
    ('Total Profit', f"${df['Profit'].sum():.0f}", '#2ecc71'),
    ('Total Orders', f"{len(df):,}", '#9b59b6'),
    ('Avg Order Value', f"${df['Sales'].mean():.1f}", '#e67e22'),
]

for i, (label, value, color) in enumerate(kpis):
    ax = fig.add_subplot(gs[0, i])
    ax.set_facecolor(color)
    ax.text(0.5, 0.65, value, transform=ax.transAxes,
           ha='center', va='center', fontsize=20,
           fontweight='bold', color='white')
    ax.text(0.5, 0.25, label, transform=ax.transAxes,
           ha='center', va='center', fontsize=11,
           color='white', alpha=0.9)
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_visible(False)

# — Sales by Category —————
ax1 = fig.add_subplot(gs[1, 0])
cat_data = df.groupby('Category')['Sales'].sum().sort_values(ascending=True)
colors_c = ['#3498db', '#e74c3c', '#2ecc71']
ax1.barh(cat_data.index, cat_data.values, color=colors_c, edgecolor='white')
ax1.set_title('Total Sales by Category', fontweight='bold', fontsize=11)
ax1.set_xlabel('Sales ($)')
ax1.xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f'${v/1000:.0f}K'))

# — Profit by Region —————
ax2 = fig.add_subplot(gs[1, 1])
reg_data = df.groupby('Region')['Profit'].sum().sort_values(ascending=False)
ax2.bar(reg_data.index, reg_data.values,
        color=sns.color_palette('pastel', 4),
        edgecolor='white', linewidth=1.5)
ax2.set_title('Total Profit by Region', fontweight='bold', fontsize=11)
ax2.set_ylabel('Profit ($)')
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f'${v:.0f}'))
```